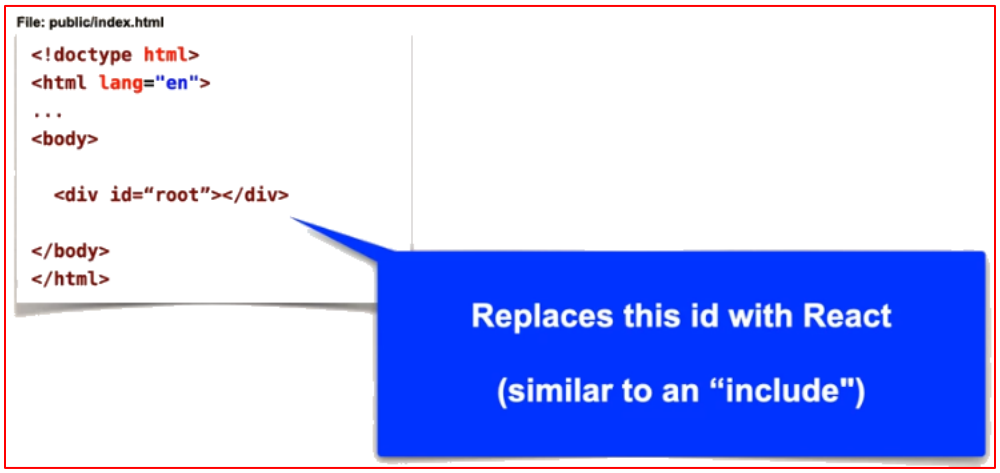
Behind the Scenes of a React Project – Overview and Demo

In this lesson, we're going to explore what happens behind the scenes when we run a React application. You might have wondered: we're using React, we have a small application, but how exactly does it load? The process involves several moving parts, but it's surprisingly digestible once we break it down.

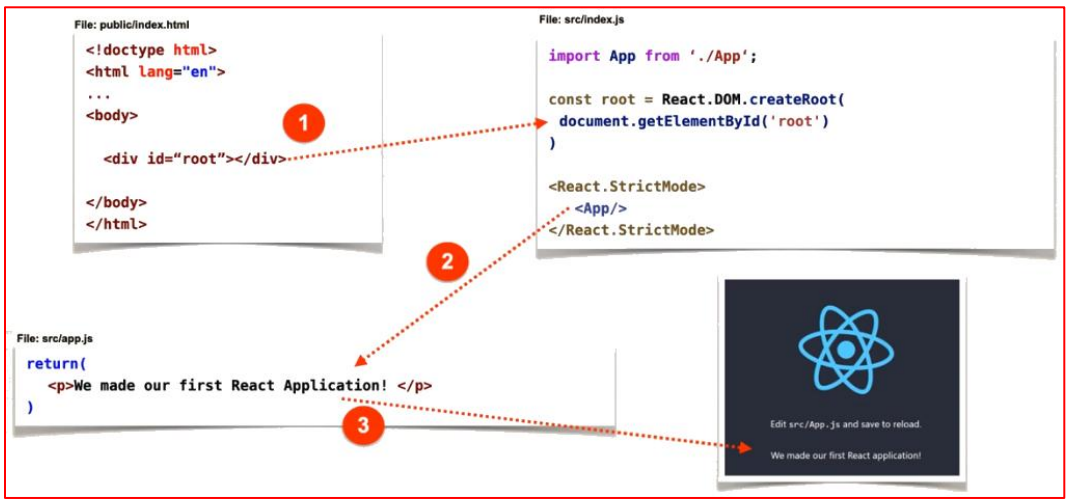
Understanding the Root File

The journey begins with our `index.html` file, which serves as the root file for the entire HTML page. This file consists of the doctype declaration, HTML tags specifying language, the body tag, and a crucial div element with the id of "root".

```
<div id="root"></div>
```



This "root" ID is vital within the `index.html` file, and here's why: we have an `index.js` file that looks specifically for the first div with an ID of "root."



Index.js and React DOM

Inside the `index.js` file, we import the app from `app.js`, then create a variable named `root`. It uses React's `createRoot` method and the JavaScript function `document.getElementById('root')` to find the first element with an ID of "root."

```
const root = React.DOM.createRoot(document.getElementById('root'));
```

What we're doing here is instructing React to search the HTML document, locate the element with an ID of "root," and replace it with our React application. Once we find "root," we implement our app component, which is wrapped within a React strict mode.

App.js and Component Rendering

Our app component returns a simple paragraph tag:

```
return <p>We made our first React application.</p>;
```

So, what happened? We began with a div possessing an ID of "root." Our `index.js` hooked onto the `index.html` to locate the "root" ID, replacing the div with our app component. The app component then returns the text, "*We made our first React application*," which renders on the screen.

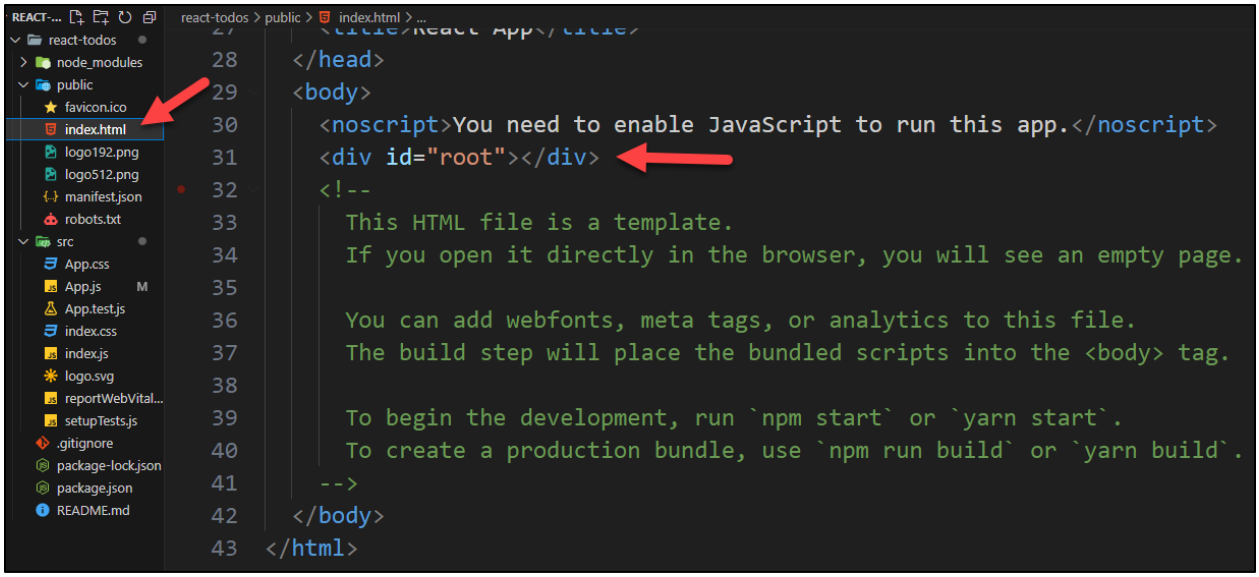
But there's more going on, like displaying the React logo and the prompt to edit source files, save, and reload. That's because there's more code above this simple return, but overall, that's precisely what transpires. We're rendering our `app.js` onto the page.

Behind-the-Scenes Investigation

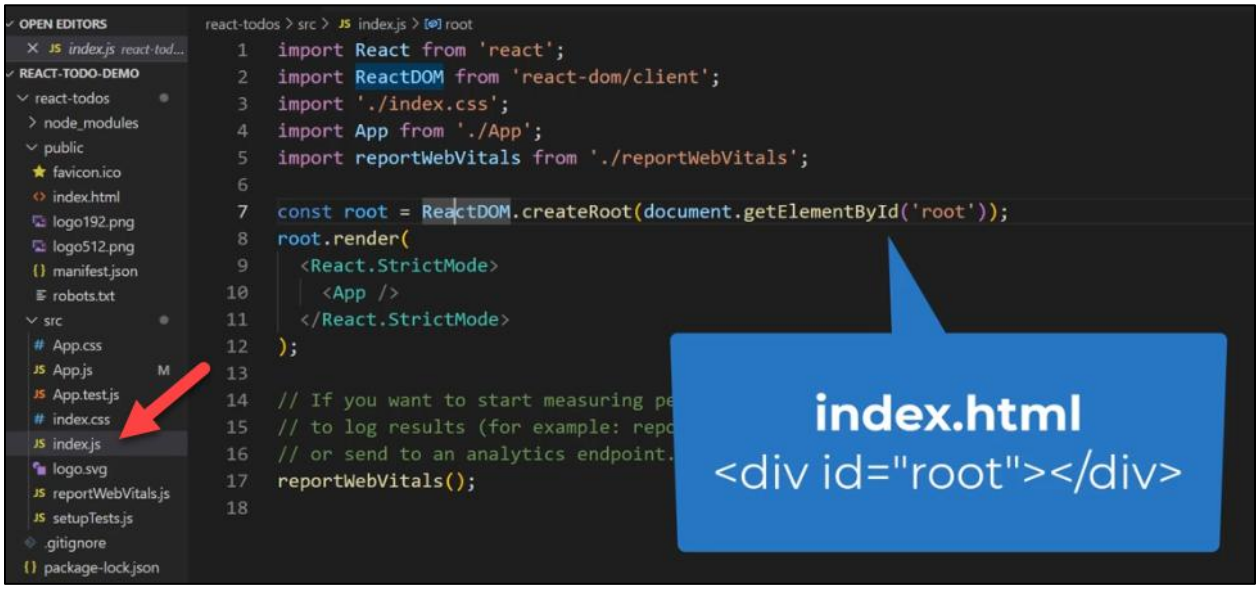
Now that we understand a bit more about the loading process, let's dive into the code and investigate behind the scenes of our React application.

Think of the `index.html` file as the global DOM – the main file that sits on the web browser itself. Here you find the doctype declaration, HTML tag, header, and a few other things. Scroll down, and you'll notice the title of your application and the body, where all the magic happens for the application.

If we look inside the body, we find an empty shell, save for a "`noscript`" tag and the div with an ID of "root." A comment even states that running this HTML page in a browser will show an empty page. But this ID "root" is doing much more than you may think.



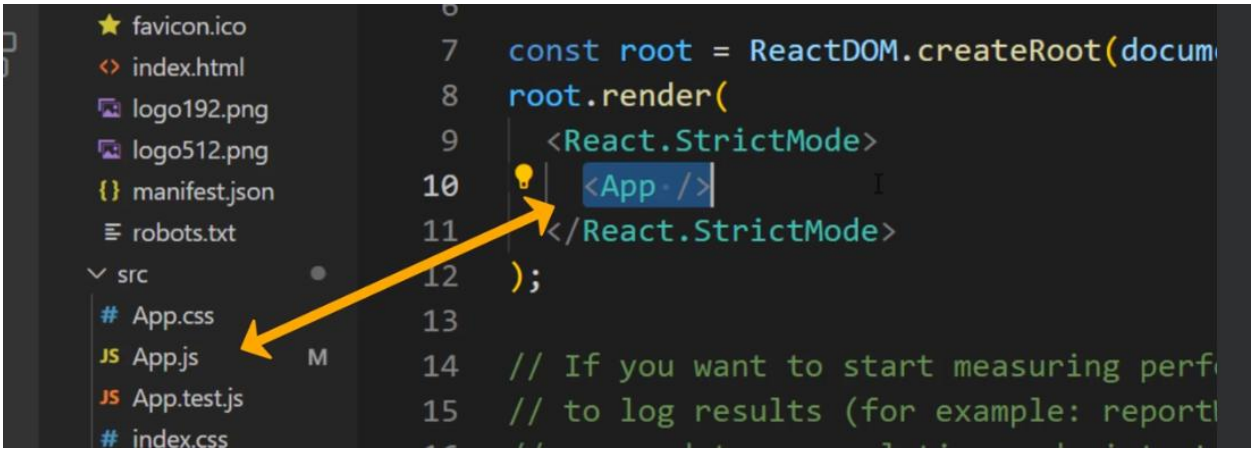
Jump over to our `index.js` file in the source directory, and you'll see imports from React, ReactDOM, index.css, app, and reportWebVitals. We have a `root` variable that equals `ReactDOM.createRoot()`, and we're passing it the div that has an ID of "root."



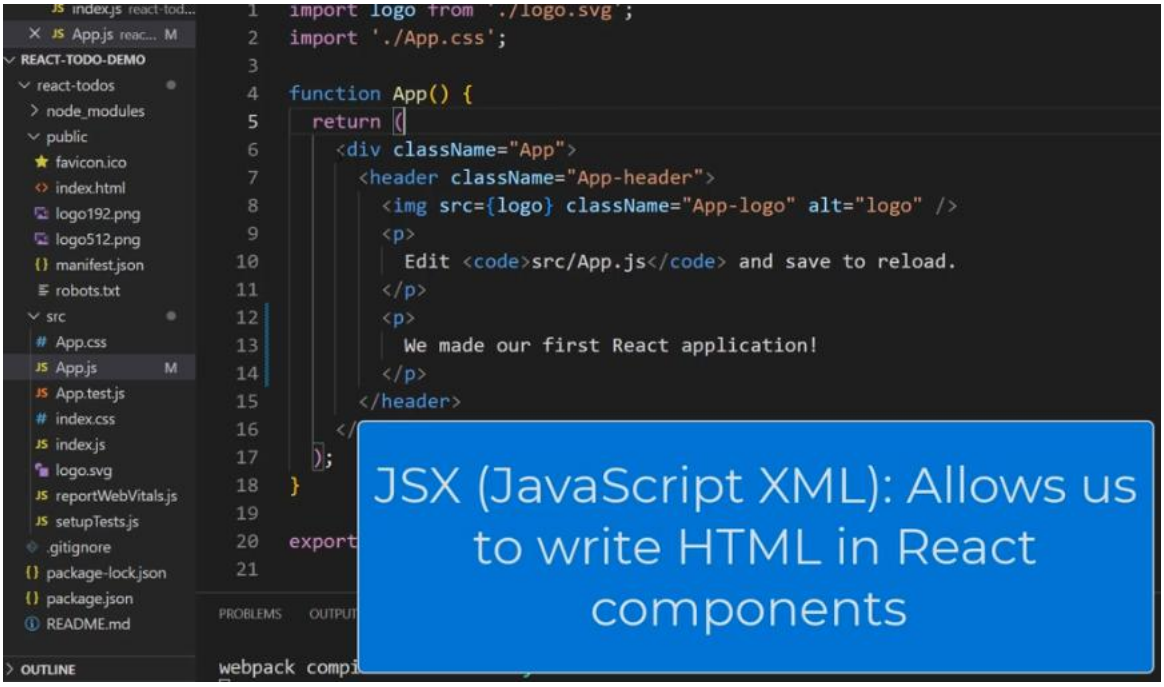
This `root` variable matches exactly with the div we just examined. After creating this React DOM root variable, we call `root.render()`, and we have our app inside. This is where we're rendering all of the components for the React application. We're hooking onto the div with the ID of "root" and creating components within that div.

The App Component

What is the app? It's a component, and that's exactly how you call components in React. The app is the `app.js` file. Inside it, you'll find imports for the logo and `.css`, and a function called "app" that returns a div.



This div might seem unusual within a JavaScript function, but React is using something called JSX. JSX is a syntax extension that allows us to write HTML-like elements within JavaScript, effectively creating DOM elements.



Within this JSX, we're instructing React to create a specific div within the div with the ID "root" on the index page. It contains various nested elements, including the React image and our new text, "We made our first application."

If we delete everything except our comment within that div, refresh the application, and open the browser, we'll see just our message. Inspect the page, and you'll find the header matching our `index.html` file, with the body containing the "noscript" tag and our div with the ID of "root."

Rendering the Components

This sequence is remarkably similar to what we examined in our `index.html` file. From here, we are instructing React to render components within our div with the ID "root," which appears as our "div class app" with the text, "We made our first React application."

In essence, this demonstrates how our `index.html` runs the entire application. We have our `index.js`, which locates the document element by the ID of "root" and renders our first component, the function app, which is running the application. This message, "We made our first React application," displays, capturing the essence of how React operates.

Now, just to clean up, you might want to remove the unnecessary import logo, but that encapsulates the behind-the-scenes workings of React.